



Biparsers: Exact Printing for Data Synchronisation

RUIFENG XIE, Peking University, China and KU Leuven, Belgium

TOM SCHRIJVERS, KU Leuven, Belgium

ZHENJIANG HU, Peking University, China

Parsers and printers are vital for data synchronisation between different serialisation formats. As they are tightly related, much research has been devoted to showing that both can be derived from a single definition. It, however, turns out to be challenging to extend this work with *exact-printing*, which recovers the original source text for the parsed data. In this paper, we propose a new approach to tackling the challenge that considers a parser-printer pair as a mechanism to synchronize the input text string with the data, and formalizes them as a bidirectional program (lens). We propose the first biparser framework to support exact-printing with non-injective parsers, provide a library of combinators for common patterns, and demonstrate its usefulness with biparsers for subsets of JSON and YAML.

CCS Concepts: • **Software and its engineering** → **Parsers; Functional languages**.

Additional Key Words and Phrases: parser combinators, exact-printing, lens, bidirectional transformations

ACM Reference Format:

Ruifeng Xie, Tom Schrijvers, and Zhenjiang Hu. 2025. Biparsers: Exact Printing for Data Synchronisation. *Proc. ACM Program. Lang.* 9, POPL, Article 74 (January 2025), 27 pages. <https://doi.org/10.1145/3704910>

1 Introduction

There are various data serialisation formats used for configuration, data transmission, and data storage. Sometimes, multiple users working on the same data might have different preferred formats. Sometimes, the users prefer to work in a different format than the supported format of their data processing tools. Sometimes, the data is modified by a program, but the user wants to preserve the formatting of the serialised document. In such cases, copies of the same data are encoded in different formats, and we need to synchronise between them, while preserving their formatting to the maximal extent. To be concrete, consider in Fig. 1 a package description that is maintained in both YAML and JSON, where a user works with the YAML file (top half) and a tool works with the JSON file (bottom half). If only the user ever edits the configuration, a YAML to JSON converter would suffice. Yet, if the tool could also edit the configuration, there should be a way to synchronise the changes to the JSON file back to the YAML file, while preserving most of its formatting.

The synchronisation problem can be broken down into designing a shared data structure T for all the formats to be synchronised, and developing a pair of parser and printer for each data format to transform between the flat textual or binary representation and T . This is the approach taken by Pandoc [MacFarlane 2024], a document converter, because writing $O(n^2)$ converters for n formats does not scale. For instance, if we want to synchronise the JSON and YAML formats, we may define the following Agda data type *Value* as a shared data structure, and write one parser-printer pair

Authors' Contact Information: Ruifeng Xie, Peking University, Beijing, China and KU Leuven, Leuven, Belgium, xieruifeng@pku.edu.cn; Tom Schrijvers, KU Leuven, Leuven, Belgium, tom.schrijvers@kuleuven.be; Zhenjiang Hu, Peking University, Beijing, China, huzj@pku.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 2475-1421/2025/1-ART74

<https://doi.org/10.1145/3704910>

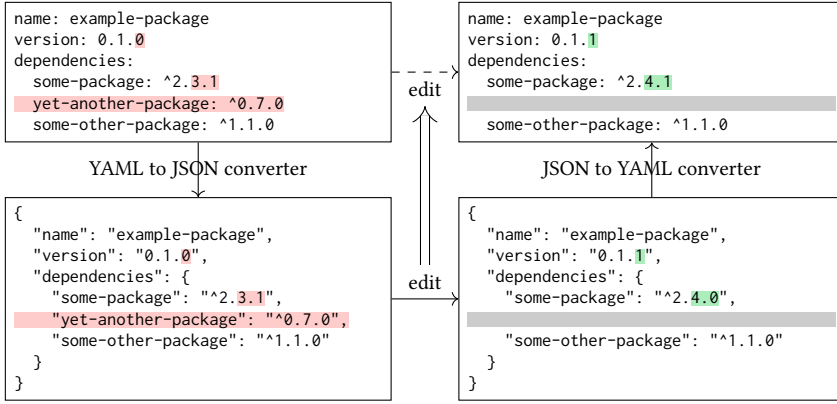


Fig. 1. JavaScript Workflow for a Package Description Maintained in Both YAML and JSON

for transformation between JSON and **Value** and another for transformation between YAML and **Value**. The **Value** type covers most common patterns in data serialisation formats: atomic values (numbers, booleans, strings, and null objects) and collections (sequences and mappings).¹

data Value : Set where

number : $(x : \mathbb{Z}) \rightarrow \text{Value}$

string : $(s : \text{String}) \rightarrow \text{Value}$

object : $(members : \text{List } (\text{String} \times \text{Value})) \rightarrow \text{Value}$

array : $(elements : \text{List Value}) \rightarrow \text{Value}$

bool : $(b : \text{Bool}) \rightarrow \text{Value}$

null : **Value**

The parser and printer are actually tightly related to each other: the generated text of a printer must be valid input for the parser, and parsing it must give back the original data that was printed. For formats intended for human reading and editing, it is also desirable for the printer to recover the exact source text (we call such a printer an *exact-printer*), so that we can edit two copies of the same data and have them synchronised without losing the formatting of either copy. However, these relations are very hard to maintain if the parser and printer are developed separately.

One possible solution is to have a single definition, an *invertible parser*, from which both parser and printer can be derived. Alimarine et al. [2005] presented bi-arrows based on arrows [Hughes 2000] as a general interface for bidirectional programming, and illustrated how parsing and pretty-printing can be defined as bi-arrows. Rendel and Ostermann [2010] use partial isomorphisms for invertible computation, and provide a generic interface for parser/printer construction. Matsuda and Wang [2018] propose a language FlipPr to program pretty-printers, from which a consistent parser can be derived. These approaches all focus on pretty-printers and leave out exact-printers, because the printer is unable to recover information about the original source text that was discarded during parsing, like whitespace, comments, and syntactic sugars.

There are two main challenges to deal with exact-printing. The first challenge lies in the *essential non-injectivity of the parser*. A parser can be considered as a (partial) function from the linear source code to the in-memory data structure, which is usually not injective. Some information in the

¹In order not to be distracted by the details of the JSON and YAML formats, we only consider a subset for each format, whose grammar is given in Figs. 2a and 2b. Details about the grammar can be found in Section 4.

source code is unnecessary for the data, and there is typically more than one textual form for every syntax tree. Unified representation for different written forms is actually beneficial because consumers of the data need not deal with those syntactic complexities and can instead focus on the semantics. Here are some typical cases of this non-injectivity.

- *Whitespace and comments are discarded.* As an example, for JSON, $\{_\text{"a"}:_\text{1}_\}$ and $\{\text{"a"}:1\}$ parse to the same [Value](#). In particular, indentations in YAML documents are discarded; only the relative indentation levels have semantic meaning, so different absolute indentations can be parsed into the same [Value](#) hierarchy.
- *The same data may have different written forms.* For YAML, we may consider `true` and `false` to be the canonical written forms for boolean values, and all other variants are syntactic sugar. Although written differently, `true` and `on` both parse to [bool true](#). For JSON, the linefeed character (U+000A) in strings can be escaped as the C-style escape sequence `\n` or the Unicode escape sequence `\u000A` (or `\u000a`).

The second challenge is *consistency validation for the printer*. The multiple written forms we mentioned above also impose difficulties for the printer. To maximally recover the original document from the data, the printer must prefer the format used in the original document, but without careful validation, the printed document might be parsed into a different value, changing the semantic meaning silently. Consider the following example involving YAML strings without quotes.

- (1) We start with a YAML document containing a single string `"abc"`;
- (2) We parse the document and get a value [string "abc"](#);
- (3) We modify the value into [string "a: bc"](#) and print it back to a YAML document;
- (4) The printer print the string verbatim using the original unquoted string format;
- (5) The final document is `"a: bc"`, which represents [object \("a" , string "bc"\) :: \[\]](#).

The problem is that printing the new string value using the unquoted string format should be invalid. The printer needs to perform some validation on the value or the printed text and reject invalid ones. However, the invalidity is not local to the printer for strings, i.e., if there were only strings but no objects, the printed text would be valid. This problem is more serious for exact-printers, because we expect exact-printers to pick up the original formatting whenever possible, while pretty-printers may choose to always use a “safe” formatting.

In this paper, we propose a new approach to tackling these challenges by considering a pair of parser and printer as a mechanism to synchronise the serialised source with the data, and formalizing them as a bidirectional program (lens) [Foster et al. 2007]. A bidirectional program defines a synchronisation between two related data structures, namely the *source* S and the *view* V , together with a *complement* C , for the information present in the source but not in the view. The forward transformation $get : S \rightarrow V \times C$ extracts the view from the source, and collects the extra information into the complement; the backward transformation $put : V \times C \rightarrow S$ then takes a modified view and the complement and produces an updated source. As far as we are aware, we are the first to provide a correct and practical lens language (bidirectional combinator language) that supports exact-printing. Our main contributions can be summarised as follows.

- *Biparsers as lenses with exact-printing.* We present our biparser framework which supports exact-printing with ASTs. Our biparsers recover the serialised source from the data, and a pair of biparsers with the same view type can be used to synchronise between two serialised copies of the same data. (Section 2)
- *Combinators for constructing biparsers.* We provide combinators for the common patterns of biparsers. All of our combinators preserve the round-trip properties, which ensures printers recover the exact input text, and parsing printed text gives back the same AST, i.e., printing does not silently change the meaning. (Section 3)

- *Implementation.* We have working biparsers for subsets of JSON and YAML, implemented in Haskell. It correctly handles whitespace, comments, multi-format data, and indentation, which illustrates the strength of our framework. (Section 4)
- *Mechanised proof in Agda.* We have proved the round-trip properties of our combinators in Agda [Norell 2007, 2009]. Most of the proof type checks in safe Agda, with one exception that we do not prove termination for the five recursive combinators (`many`, `some`, `many'`, `some'`, and `sepBy1`). For the details, please refer to the relevant code.

This paper is a literate Agda program with the proofs stripped. The complete Agda proof, the Haskell library with the examples for subsets of JSON and YAML, and the appendix of this paper are available in the artifact [Xie et al. 2024]. We try our best to avoid advanced features of Agda, so that readers who are familiar with Haskell should also be able to read the definitions. For the Agda-specific features we use to improve readability (e.g., copatterns), we provide explanations when they are first used. We also provide a summary in the appendix.

2 Biparsers as Lenses

As stated in Section 1, parsers are generally non-injective, so the exact-printer needs extra information which is stored as part of the syntax tree. This observation fits nicely into the lens structure (bidirectional transformation): in the general sense, the biparsers we propose are lenses between the source text (source) and the data (view), and the extra information, such as whitespace, parentheses, different written forms of the same data, etc., are stored separately in the *complement*.

In this section, we discuss our definition of biparsers and how they make use of the complements to achieve exact-printing, and clarify the round-trip properties as fundamental correctness properties for biparsers.

2.1 Definition of Biparser

A biparser consists of a parser and a printer. We capture the effects of parsing and printing both using the same `P` monad, a familiar monad for parsers used in various parser combinator frameworks [Leijen and Meijer 2001]. The `P` monad supports error reporting for a specific error type `E`, and manipulation of an internal string buffer of type `List Token` with some user state `U`. The user state `U` is some state that changes alongside the internal string buffer; in our example YAML biparser (Section 4.2), we use this `U` to track the indentation stack. In this paper, we fix the token type to `Char` and the error type to `String` for simplicity, but it is easy to use other types.

```
record P (Token U E A : Set) : Set where
  constructor mkP
  field runP : List Token × U → Result E (A × List Token × U)
```

The `P` monad provides the functorial mapping `<$>`, the monadic operations `pure` and `>=>`, and also a choice operator `<|>` for backtracking, with their usual definitions; the type `Result E A` is the sum of `ok a` for success and `err e` for failure. Refer to the artifact for full definitions.

A parser works by consuming a prefix of the input string, producing an output value `A`, and passing on the remainder of the input string. As the dual of parsers, our printers work in the opposite direction, prepending to the input string and producing an output value `A`; it may also report an error. Therefore, when a printer is about to print a certain fragment of the syntax tree, the text following that fragment is already given as the input string. We exploit this property for validity checks in the `notFollowedBy` combinator in Section 3.3.

Based on the `P` monad, we can define our biparser as a pair of parse and print functions. A biparser operates between an input `X` and an output `Y`, with a complement `C`:

```

record Biparser (Token U E C X Y : Set) : Set where
  constructor mkBiparser
  field parse : X → P Token U E (Y × C)
  print : (Y × Maybe C) → P Token U E X

```

For top-level biparsers, the input X is usually unit ($\top$) and the output Y is usually the data type. X is unit because their parser does not require an input and their printer does not produce an output. However, this parameter X is still necessary because this Kleisli-arrow-based representation resembles that of lenses and is more convenient for composition.

In the printer, the input complement is optional (wrapped in `Maybe`). This allows printing data without an existing complement. Since the complement represents extra information from the previous version of the source text, when we are generating the source text from scratch, we have no complement and must pass in `nothing`. Equivalently, we may split the `print` above into the following two functions:

```

printWithComplement : (Y × C) → P Token U E X
printWithoutComplement : Y → P Token U E X

```

which correspond to the components $put : Y \times C \rightarrow X$ and $create : Y \rightarrow X$ for pure lenses.

2.2 Round-Trip Properties

Round-trip properties are the fundamental correctness properties for bidirectional programs. Compared to lenses, they are more complicated for biparsers: a biparser transforms between the input X and the output-complement pair $Y \times C$, just like lenses, but it also manipulates the internal string buffer and the user state, and it optionally raises an error. The round-trip properties must be extended accordingly.

We start by considering the manipulation of the internal state $IS = \text{List Token} \times U$. If we expand the definition of the `P` monad, a biparser is essentially a pair of partial functions between $IS \times X$ and $IS \times Y \times C$. Therefore, we can derive the round-trip properties of a biparser from a lens with $IS \times X$ as the source, $IS \times Y$ as the view, and C as the complement.

The next step is to figure out how the failures of the parser and the printer are related. Intuitively, if parsing an input string succeeds, then it means there is at least one valid textual representation (i.e., the original input text) of the resulting data, so printing that data should also succeed. The other way around, if printing the data succeeds, then there is at least one valid parse of the produced string, so parsing that string should also succeed. Therefore, we require that the success of one direction should imply the success of the other.

Combining the above intuitions, we specify the round-trip properties as follows. For any biparser $p : \text{Biparser Token U E C X Y}$, these two properties must be true:

```

runP (parse p x) s ≡ ok ((y, c), s') ⇒ runP (print p (y, just c)) s' ≡ ok (x, s)    (PARSEPRINT)
runP (print p (y, mc)) s ≡ ok (x, s') ⇒ ∃c'. runP (parse p x) s' ≡ ok ((y, c'), s)    (PRINTPARSE)

```

Notice that just like for lenses, even if the complement is provided (i.e., $mc = \text{just } c$), `PRINTPARSE` does not require that the complement is recovered (parse after print might give a different c' instead of the original c), because the information in the complement is not always fully included in the printed source text. A typical example of this situation is biparsers for data with multiple written formats. Consider the following example:

Example 2.1. Inside double-quoted YAML strings, a character c may be written in one of the following formats, depending on the specific character:

- (1) The character c itself, if it is not a control character;

- (2) Escaped ASCII control characters, i.e., `\n`, `\t`, etc., if it is one of the special characters;
- (3) Escaped 8-bit Unicode, i.e., `\x` followed by two hexadecimal digits, if representable;
- (4) Escaped 16-bit Unicode, i.e., `\u` followed by four hexadecimal digits, if representable;
- (5) Escaped 32-bit Unicode, i.e., `\U` followed by eight hexadecimal digits;

A natural choice for the complement is the sum of all the above format possibilities. However, some view-complement pairs are not possible output of the parser: e.g., most characters cannot be encoded using format (2), and the smiling face emoji (U+1F600) requires more than four hexadecimal digits, so it is never encoded in formats (3) and (4). Yet, even if the printer is requested to print U+1F600 with a complement indicating format (3), it is still desirable to have the printer print valid source code by ignoring the complement and falling back on format (5). In such cases, printing the data forces the use of a different format from the complement, so the complement cannot be recovered by a print-parse round-trip.

In general, failure to recover the complement is a result of the view-complement pair $Y \times C$ having more information than the source X and the input text segment S combined. Introducing the complement C makes the mapping from $X \times S$ to $Y \times C$ injective, but it is not always possible to also make that mapping surjective.

The **PARSEPRINT** property only specifies the behaviour for a biparser when the view remains unchanged. One would also like it to be the case that making a “small” change to the view would have “small” local changes on the source and the text. However, the source and the view are user-defined data structures, and the biparser is also implemented by the user (where different primitives and combinators denote different update strategies), so it is hard to formalise this “small change” property in general. We can only provide some informal reasoning: even when the view is modified, we can usually still expect the printer to recover the unchanged parts. Parser combinators encourage writing compositional programs, and large biparsers usually consist of smaller biparsers. As a result, the round-trip properties also apply to these sub-biparsers, associating unchanged subtrees in the AST with the corresponding input string segment. For example, the biparser $p \llcorner q$ (Section 3.2) has the nice property that changes to the second component of the view have no impact on the first component. For recursive biparsers like **many** and **some**, we can also minimise the damage by aligning views and complements using an appropriate custom alignment strategy (Section 3.4).

2.3 Identity and Composition

Similar to lenses, biparsers have a composition operator with an identity. This identity biparser trivially satisfies the round-trip properties, while the composition operator preserves them.

The identity biparser synchronises between the same input and output type X with a unit complement, and leaves the string state untouched.

```
id : Biparser Token U E  $\top$  X X
runP (parse id x) s = ok ((x , tt) , s)
runP (print id (x , _)) s = ok (x , s)
```

For better readability, we use *copatterns* to define biparsers. This means a value of the **Biparser** type is defined by giving separate definitions to the **parse** and **print** fields. We do the same for the **P** type. As an example, the above definition of **id** can be equivalently written using constructors as follows:

```
id' = mkBiparser ( $\lambda$  x  $\rightarrow$  mkP ( $\lambda$  s  $\rightarrow$  ok ((x , tt) , s)))
      ( $\lambda$  (x , _)  $\rightarrow$  mkP ( $\lambda$  s  $\rightarrow$  ok (x , s)))
```

Notice how **parse** and **print** correspond to **mkBiparser**, and **runP** corresponds to **mkP**.

The composition chains two biparsers, with the output of the first parser being the input of the second parser. The composed biparser uses a pair of complements from both component parsers.

```

_•_ : Biparser Token U E C2 Y Z
    → Biparser Token U E C1 X Y
    → Biparser Token U E (C1 × C2) X Z
parse (f • g) x = do
  (y, c1) ← parse g x
  (z, c2) ← parse f y
  pure (z, (c1, c2))
print (f • g) (z, mc) = do
  y ← print f (z, proj2 ⋈ mc)
  print g (y, proj1 ⋈ mc)

```

The Agda syntax `_op_` defines a binary operator `op`, such that $(x \text{ op } y) \equiv (_op_ x \ y)$; operators can also be partially applied, such that $(x \text{ op } _) \equiv (_op_ x)$ and $(_op \ y) \equiv (\lambda x \rightarrow _op_ x \ y)$. The identity biparser `id` is the left and right identity for biparser composition, up to the complement isomorphism that unit `⊤` is the left and right identity of products.

3 Biparser Combinators

In this section, we introduce biparser combinators, generalised from arrows [Hughes 2000] and bi-arrows [Alimarine et al. 2005]. These combinators capture common bi-parsing patterns and come with verified round-trip properties. This way larger biparsers for specific languages can be composed from these combinators, and the round-trip properties are automatically guaranteed by construction. We classify the combinators into primitive combinators and derived combinators: primitive combinators rely on the internals of biparsers, and their round-trip properties are verified manually; on the other hand, derived combinators are composed from other combinators, and their round-trip properties follow automatically.

Wadler [1995] presented a concise collection of primitive parsers and combinators for constructing recursive descent parsers. At the core of this collection is a primitive parser named *item* for consuming a character from the input stream. On top of that, there are four ways to combine small parsers into larger ones: sequencing, alternation (choice), iteration, and filtering. Moreover, although not used by Wadler, parse results can be transformed using *fmap* from the *Functor* interface. We follow this structure and provide combinators for each of these purposes.

3.1 Primitive Biparser Combinators

We start with primitive biparsers, the self-contained building blocks to construct larger biparsers. They take no other biparser as input.

End of input. We define a primitive biparser `eof` for end of input. To satisfy the `PRINTPARSE` property, both the parser and the printer need to check that the string buffer is empty.

```

eof : Biparser Token U String ⊤ ⊤ ⊤
runP (parse eof tt) ([], u) = ok ((tt, tt), [], u)
runP (parse eof tt) (x :: xs, u) = err "eof.parse: expecting eof"
runP (print eof _) ([], u) = ok (tt, [], u)
runP (print eof _) (x :: xs, u) = err "eof.print: expecting eof"

```

Token consumption. We define a primitive biparser `item` that consumes a single character. The forward semantics (i.e., parsing) can be directly taken from real-world parser combinator libraries [Leijen and Meijer 2001], and the backward semantics can be designed correspondingly.

```

item : Biparser Token U String  $\top$   $\top$  Token
runP (parse item tt) ([], u) = err "item.parse: unexpected eof"
runP (parse item tt) (x :: xs, u) = ok ((x, tt), (xs, u))
runP (print item (x, _)) (xs, u) = ok (tt, (x :: xs, u))

```

Transforming results. Parser results are usually characters and strings, which need to be transformed into various structures in the ASTs. For unidirectional parsers, this is supported by providing a *Functor* interface for parsers. For biparsers, the transformation between the actual output X of a biparser and the desired output type Y needs to be bidirectional. In fact, because biparsers are arrows, we can express the transformation itself as a `Biparser` between X and Y . The transformation can be combined with the biparser simply using biparser composition.

Specifically, we can embed isomorphisms and *partial isomorphisms* [Rendel and Ostermann 2010] (or equivalently, *affine traversals* [Clarke et al. 2022]) into biparsers. A partial isomorphism consists of a pair of partial functions, so we only need to wrap them and pass on the input string state.

```

partialIso : (to : X  $\rightarrow$  Result E Y)  $\rightarrow$  (from : Y  $\rightarrow$  Result E X)
             $\rightarrow$   $\llbracket$  IsP Iso to from  $\rrbracket$   $\rightarrow$  Biparser Token U E  $\top$  X Y
parse (partialIso to from) x = (_, tt)  $\triangleleft$  (liftError (to x))
print (partialIso to from) (y, _) = liftError (from y)

```

where `liftError` lifts the result of the partial functions into the `P` monad:

```

liftError : Result E X  $\rightarrow$  P Token U E X
runP (liftError (err e)) _ = err e
runP (liftError (ok x)) s = ok (x, s)

```

For the round-trip properties of `partialIso`, the two functions *to* and *from* must form a partial isomorphism, i.e., it must be true that $to\ x \equiv ok\ y$ is logically equivalent to $from\ y \equiv ok\ x$. This requirement is captured in the `IsP Iso` record, passed as an implicit instance argument to `partialIso` (marked by “ $\llbracket$ ” and “ $\rrbracket$ ”). It is there only for the proof, so readers may safely ignore it.

Then, an isomorphism is just a special case of partial isomorphisms:

```

iso : (to : X  $\rightarrow$  Y)  $\rightarrow$  (from : Y  $\rightarrow$  X)  $\rightarrow$   $\llbracket$  Is Iso to from  $\rrbracket$   $\rightarrow$  Biparser Token U E  $\top$  X Y
iso to from = partialIso (ok  $\circ$  to) (ok  $\circ$  from)

```

where the two functions $to : X \rightarrow Y$ and $from : Y \rightarrow X$ must form an isomorphism (`Is Iso`).

Example 3.1 (Constructors). Parse results need to be wrapped in various constructors of the data type: for example, in our `Value` type for JSON and YAML, every number literal n is represented as `number n : Value`. For each constructor $C_k : X_1 \rightarrow X_2 \rightarrow \dots \rightarrow X_n \rightarrow T$ of type `T`, we can mechanically generate the following biparser as a partial isomorphism:

```

mkCk : Biparser Token U String  $\top$  (X1  $\times$  X2  $\times$   $\dots$   $\times$  Xn) T
mkCk = partialIso wrap unwrap
  where wrap : X1  $\times$  X2  $\times$   $\dots$   $\times$  Xn  $\rightarrow$  Result E T
        wrap (x1, x2, ..., xn) = ok (Ck x1 x2 ... xn)
        unwrap : T  $\rightarrow$  Result String (X1  $\times$  X2  $\times$   $\dots$   $\times$  Xn)
        unwrap (Ck x1 x2 ... xn) = ok (x1, x2, ..., xn)

```



```
unwrap _ = err "expecting constructor Ck"
```

This biparser is similar to the *prisms* generated by *makePrisms* in the *lens* library [Kmett 2024].

Example 3.2 (Assigning constant values). Sometimes, we want to assign a constant value as the output of a biparser. For example, `null` in a JSON document should be parsed into the `Value null`. For this purpose, we have the following combinator:

```
const : (x : X) (p : Decidable (x ≡ _)) → Biparser Token U String T T X
const x p = partialIso (λ _ → ok x) (λ x' → if isYes (p x') then ok tt else err "unexpected")
```

The predicate p can be simply understood to have type $X \rightarrow \text{Bool}$, checking for equality to the distinguished value x . If X has decidable equality in general, p can be derived accordingly:

```
const' : {X} Eq X → X → Biparser Token U String T T X
const' x = const x (λ x' → x == x')
```

Here the constraint $\{ \text{Eq } X \}$ is analogous to the Haskell type class constraint “ $\text{Eq } x \Rightarrow$ ”.

Transforming the complements. The complement of any biparser can be transformed arbitrarily by an injection. This fact is demonstrated by the following biparser combinator:

```
mapC : (to : C1 → C2) → (from : C2 → C1) → {Inv to from}
      → Biparser Token U E C1 X Y → Biparser Token U E C2 X Y
parse (mapC to from p) x = (λ (x , c) → (x , to c)) <#> (parse p x)
print (mapC to from p) (y , mc2) = print p (y , from <#> mc2)
```

For the round-trip properties of `mapC`, the transformation functions $\text{to} : C_1 \rightarrow C_2$ must be an injection, with $\text{from} : C_2 \rightarrow C_1$ as its left-inverse (i.e., $\text{from} \circ \text{to} = \text{id}$). Recovery of the complement is not required for the *from-to* round-trip. This is a direct result of `PRINTPARSE`’s requirements.

This combinator can be used to make the complement more readable. For example, it can be used to eliminate the T types in the complement of the biparser composition $f \bullet g$, if either f or g has a T complement.

Example 3.3 (Variants of the composition operator). For convenience and readability, we define the following two variants of biparser composition:

```
_#_ : Biparser Token U E T Y Z → Biparser Token U E C X Y → Biparser Token U E C X Z
f # g = mapC proj1 (λ , tt) (f • g)
```

```
_•#_ : Biparser Token U E C Y Z → Biparser Token U E T X Y → Biparser Token U E C X Z
f •# g = mapC proj2 (tt , λ) (f • g)
```

The $\#$ sign in the operator indicates that the biparser on that side has a discarded T complement.

Storing extra information in the complement. It is also possible to store some redundant information about the view in the complement, which will be very useful when we want to perform view-complement alignment (Section 3.4). We have the following combinator:

```
store : (X → C) → Biparser Token U E C X X
parse (store f) x = pure (x , f x)
print (store f) (x , _) = pure x
```

This also illustrates that the complement need not be recovered in a print-parse round-trip.

Making parts of the complement optional. The printer `print` for biparsers takes a `Maybe c` so that data can still be printed even if we do not have a suitable complement yet. However, it is an all-or-nothing situation: either we have the whole complement `c` and we pass in `just c`, or we provide no complement at all and pass in `nothing`.

The following combinator wraps the complement of a biparser into a sum type, making it inherently optional. When the resulting biparser is composed with other biparsers, we have a finer-grained control over the complement.

```
relaxCL : Biparser Token U E C1 X Y → Biparser Token U E (C1 ∪ C2) X Y
parse (relaxCL p) x = (λ (y , c) → (y , inj1 c)) <#> parse p x
print (relaxCL p) (y , mc) = print p (y , join (either just (λ _ → nothing) <#> mc))
```

`relaxCL` injects the complement into the left branch. Since sums are symmetrical, we can use the `mapC` combinator to transform the complement type and get the following `relaxCR` combinator:

```
relaxCR : Biparser Token U E C2 X Y → Biparser Token U E (C1 ∪ C2) X Y
relaxCR p = mapC (either inj2 inj1) (either inj2 inj1) (relaxCL p)
```

Observe that `Maybe C` is isomorphic to `C ∪ ⊤`, so we also have the following specialised version:

```
relaxC : Biparser Token U E C X Y → Biparser Token U E (Maybe C) X Y
relaxC p = mapC (either just (λ _ → nothing)) (maybe inj1 (inj2 tt)) (relaxCL p)
```

This will be useful when we consider repetition, choice, and alignments.

Validation. Using `partialIso`, we can define a biparser which validates that the value `x : X` passed through satisfies a particular predicate `p : X → Bool`:

```
validate : (X → Bool) → Biparser Token U String ⊤ X X
validate p = partialIso check check
  where check = λ x → if p x then ok x else err "invalid value"
```

Notice that the predicate `p` is also verified in the printer. If values not satisfying `p` are not properly rejected in the printer, the biparser would violate the `PRINTPARSE` property. This verification is in turn enforced by the requirements of `partialIso`: if the backward transformation *from* succeeds, the forward transformation *to* must also succeed.

We may compose `item` and `validate` for validated token consumption:

```
satisfy : (Token → Bool) → Biparser Token U String ⊤ ⊤ Token
satisfy p = validate p # item
```

Biparsers like `satisfy` are derived by combining other primitive biparsers and biparser combinators. Therefore, we get the round-trip properties for `satisfy` for free.

For another example which both transforms and validates, consider parsing a digit as an integer. The unidirectional definition (indicated by the subscript U) uses the operator `_<#>_`:

```
digitU = toDigit <#> satisfyU isDigit
```

The `toDigit : Char → Int` function is partial, but we can make it total by having it return zero for non-digit characters without changing the meaning of this parser, because we have verified the character with `isDigit`. In comparison, for biparsers, the function `toDigit` is replaced with a partial isomorphism, and `fmap` is replaced with biparser composition:

```
digit = partialIso toDigit' fromDigit' # satisfy isDigit
```

Here `toDigit' : Char → Result String ℕ` and `fromDigit' : ℕ → Result String Char` form a partial isomorphism between digit characters and their integer value. The structural difference of `digitU`

and `digit` is a consequence of monads/applicatives v.s. arrows style. From a monadic point of view, parsers and transformations are distinguished, but in our arrow-style theory, a parser is just a special transformation from the unit input $\top$. The check in `satisfy isDigit` is actually redundant, because we need to do this check in the partial isomorphism anyway. Therefore, `item` in place of `satisfy isDigit` also works. We keep it this way to make the comparison clear.

3.2 Sequencing

Biparser composition $f \bullet g$ works like a pipeline: f takes the output of g and applies further transformations to it. Similar to biparser compositions, biparser sequencing performs the parser and printer effects of both f and g in sequence, but unlike biparser compositions, it pairs the input and the output of f and g . In other words, it constructs biparsers for product types.

Existing frameworks implement *Applicative* (and perhaps also *Monad*) for sequencing. Unfortunately, as observed in *Invertible Syntax Descriptions* [Rendel and Ostermann 2010], the *Applicative* ($\langle * \rangle$) operator cannot be easily extended to support bidirectional programs, because it relies on currying unidirectional functions. The workaround for bidirectional programs is to pair the arguments before applying them to an uncurried function, instead of applying the arguments one by one to a curried function. For example, consider the following chain application:

$$f \langle \> x_1 \langle * \rangle x_2 \langle * \rangle \dots \langle * \rangle x_n$$

For unidirectional parser combinators, both $_ \langle \> _$ and $_ \langle * \rangle _$ are left-associative, and the transformation function $f : X_1 \rightarrow X_2 \rightarrow \dots \rightarrow X_n \rightarrow Y$ is applied left-to-right to each $x_k : \text{Parser } X_k$:

$$((f \langle \> x_1) \langle * \rangle x_2) \langle * \rangle \dots \langle * \rangle x_n$$

In comparison, for *Invertible Syntax Descriptions*, the $_ \langle * \rangle _$ operator associates to the right and have a different type, which is a generalisation to the $_ \star _$ operator for applicative functors as *lax monoidal functors* [McBride and Paterson 2008]:

$$f \langle \> (x_1 \star (x_2 \star \dots \star x_n))$$

Therefore, the arguments $x_k : \text{Parser } X_k$ are first paired into $\text{Parser } (X_1 \times (X_2 \times \dots \times X_n))$ and then applied to $f : \text{Iso } (X_1 \times (X_2 \times \dots \times X_n)) \rightarrow Y$, a partial isomorphism. In our theory, biparsers also serve as the transformation, so the final application is achieved by biparser composition, and we only need the biparser counterpart of $_ \langle * \rangle _$ for sequencing.

We define the sequencing $_ *** _$ for biparsers as a generalisation of the product combinator for bi-arrows. It suffices to define the following `first` combinator which transforms the first part of the input (from X_1 to X_2 , using a biparser p) and passes on the second part (Y):

```
first : Biparser Token U E C X1 X2 → Biparser Token U E C (X1 × Y) (X2 × Y)
parse (first p) (x1, y) = (λ (x2, c) → ((x2, y), c)) <> (parse p x1)
print (first p) ((x2, y), c) = (λ x1 → (x1, y)) <> (print p (x2, c))
```

Based on `first`, we derive a similar combinator `second` that transforms the second part instead:

```
second : Biparser Token U E C Y1 Y2 → Biparser Token U E C (X × Y1) (X × Y2)
second p = swap # first p # swap
where swap : Biparser Token U E C (X × Y) (Y × X)
      swap = iso (λ (x, y) → (y, x)) (λ (y, x) → (x, y))
```

Then we obtain $_ *** _$ by composing `first` and `second`:

```
_***_ : Biparser Token U E C1 X1 Y1
      → Biparser Token U E C2 X2 Y2
```

$$\begin{aligned} &\rightarrow \text{Biparser Token } U E (C_1 \times C_2) (X_1 \times X_2) (Y_1 \times Y_2) \\ f *** g &= \text{second } g \bullet \text{first } f \end{aligned}$$

This construction is similar to arrows [Hughes 2000] and bi-arrows [Alimarine et al. 2005]. Our addition, the complements of f and g , are automatically handled by the biparser composition. The *id* biparser, with $\top$ as the input and the output, is the left and right identity for $***$, up to the isomorphism that $\top$ is the left and right identity for product types.

For parsers with unit input, we have the following specialisation:

$$\begin{aligned} _ \langle\!\langle * \rangle\!\rangle _ &: \text{Biparser Token } U E C_1 \top X \\ &\rightarrow \text{Biparser Token } U E C_2 \top Y \\ &\rightarrow \text{Biparser Token } U E (C_1 \times C_2) \top (X \times Y) \\ f \langle\!\langle * \rangle\!\rangle g &= (f *** g) \bullet^\# \text{dupl} \\ \text{where dupl} &= \text{iso } (\lambda (tt) \rightarrow (tt, tt)) (\lambda (tt, tt) \rightarrow tt) \end{aligned}$$

Here, the isomorphism *dupl* serves to eliminate the extra $\top$ in the input, since $\top$ is only the unit of products up to an isomorphism. This pattern will show up several more times in later sections.

There are always situations where some parsed information is not directly present in the syntax trees. We can *drop* the output of a parser into the complement and recover it later in the printer:

$$\begin{aligned} \text{drop} : X &\rightarrow \text{Biparser Token } U E X X \top \\ \text{parse } (\text{drop } x_0) \ x &= \text{pure } (tt, x) \\ \text{print } (\text{drop } x_0) \ (tt, x) &= \text{pure } (\text{fromMaybe } x_0 \ x) \end{aligned}$$

We also provide the following convenient operators. Combined with *drop*, they serve the same purposes as $(\gg)$ and $(\ll)$ for applicative functors.

$$\begin{aligned} _ \gg _ &: \text{Biparser Token } U E C_1 \top \top \rightarrow \text{Biparser Token } U E C_2 X Y \\ &\rightarrow \text{Biparser Token } U E (C_1 \times C_2) X Y \\ _ \ll _ &: \text{Biparser Token } U E C_1 X Y \rightarrow \text{Biparser Token } U E C_2 \top \top \\ &\rightarrow \text{Biparser Token } U E (C_1 \times C_2) X Y \end{aligned}$$

They can be defined by composing the two biparsers with $_ \langle\!\langle * \rangle\!\rangle _$, and applying a simple isomorphism on the result, dropping the units. Similar to Example 3.3, we can define variants of these sequential composition operators by wrapping the result biparser with *mapC*: $_ \# \langle\!\langle * \rangle\!\rangle _$, $_ \langle\!\langle * \rangle\!\rangle \# _$, $_ \# \ll _$, and $_ \ll \# _$.

Example 3.4 (Quotation marks for string literals). Assume that we have a biparser *inner* for inner contents of a string literal, and a biparser *doubleQuote* for quotation marks.

$$\begin{aligned} \text{doubleQuote} &: \text{Biparser Char } U E \top \top \top \\ \text{inner} &: \text{Biparser Char } U E C X Y \end{aligned}$$

We may surround *inner* with a pair of *doubleQuotes* to get the biparser for string literals:

$$\begin{aligned} \text{stringLiteral} &: \text{Biparser Char } U E C X Y \\ \text{stringLiteral} &= \text{doubleQuote } \# \gg \text{inner } \ll \# \text{doubleQuote} \end{aligned}$$

The quotation marks do not contribute to the data.

Example 3.5 (Trailing whitespace). Assume that we have a biparser *spaces* for whitespace, and an arbitrary biparser p . We can combine them to get a biparser q taking care of the trailing whitespace:

$$\begin{aligned} \text{spaces} &: \text{Biparser Char } U E \top \top (\text{List Char}) \\ p &: \text{Biparser Char } U E C X Y \end{aligned}$$

Trailing whitespace is saved to the complement by `drop`, and combined with p using `_<*_`:

```
q : Biparser Char U E (C × List Char) X Y
q = p <*_ (drop [] •# spaces)
```

We need to make a choice about the default trailing whitespace string, because it is required as an argument of `drop`. The empty string (`[]`) is chosen here for simplicity.

The `drop` combinator saves and recovers some value using the complement. However, if the value is always a fixed constant, it is better to discard it right away. Therefore, we define the following `char` combinator for parsing a known character, different from `satisfy` ($c == _$):

```
char : { Eq Token } → Token → Biparser Token U String T T T
char c = partialIso eqC (λ (tt) → ok c) •# item
  where eqC = λ x → if x == c then ok tt else err "unexpected character"
```

The output of `char` is `T`, instead of `Char` as in unidirectional parser combinator frameworks. This is a subtle difference between the design of bidirectional programs and unidirectional programs: superfluous information cannot be easily discarded, and may lead to complication.

Similarly, we can define the `string` combinator for parsing a known list of characters:

```
string : { Eq Token } → List Token → Biparser Token U String T T T
string [] = id
string (c :: cs) = char c #> string cs
```

3.3 Choice

There are two different variants of choices. The first variant maps between two sum types, so the choice of branches is deterministic according to the input and output data structures. The second variant involves two fallible mappings, which are attempted one after the other with backtracking, and the first successful attempt is taken as the final result. Backtracking choices in turn have two different variants, the normal biased choice and the remembered choice.

3.3.1 Deterministic Choice. The deterministic choice is similar to the bifunctor mapping for sum types. It is captured by the `_++_` combinator, generalising from the corresponding bi-arrow combinator:

```
_++_ : Biparser Token U E C X1 Y1
      → Biparser Token U E C X2 Y2
      → Biparser Token U E C (X1 ⊔ X2) (Y1 ⊔ Y2)
parse (p ++ q) (inj1 x1) = (λ (y1, c) → (inj1 y1, c)) <#> (parse p x1)
parse (p ++ q) (inj2 x2) = (λ (y1, c) → (inj2 y1, c)) <#> (parse q x2)
print (p ++ q) (inj1 y1, c) = inj1 <#> (print p (y1, c))
print (p ++ q) (inj2 y2, c) = inj2 <#> (print q (y2, c))
```

The deterministic structure ensures that the same biparser p or q is used throughout any successful round-trip, so the round-trip properties are preserved.

Notice that both branches share the same complement type C , because we want to support, for example, modifying the output from `inj1 y1` to `inj2 y2`. In that case, the complement was produced by `parse p` (because the original output was `inj1 y1`), but it would be consumed by `print q` (because the output is modified to `inj2 y2`), and the only way this is possible is to have the same complement type for both branches. However, we can circumvent this restriction by unifying the complement types with the `mapC`, `relaxCL`, and `relaxCR` combinators. See `_<+>_` below for an example.

Similar to sequencing, the following **void** biparser, with $\perp$ as the input and the output, is the left and right identity for deterministic choices, up to the isomorphism for input and output, that $\perp$ is the left and right identity for sum types.

```
void : Biparser Token U E C  $\perp$   $\perp$ 
parse void ()
print void () , _)
```

This biparser **void** serves as an impossible branch because $\perp$ is uninhabited.

3.3.2 Biased Choice. Biased choice is a variant of backtracking choice generalised from the unidirectional *Alternative* interface for the **P** monad. It attempts the left branch first and falls back on the right branch in case of failure, so if both branches would succeed, the left branch is preferred.

```
_<+>_ : (p q : Biparser Token U E C X Y)
  →  $\llbracket$  ParsePrintConsistent p q  $\rrbracket$ 
  →  $\llbracket$  PrintParseConsistent p q  $\rrbracket$ 
  → Biparser Token U E C X Y
parse (p <+> q) x = parse p x <|> parse q x
print (p <+> q) yc = print p yc <|> print q yc
```

In order to support the fallback, the information which branch was taken in either **parse** or **print** is not reflected in the input or the output using a sum type. Instead, both branches must share the same X and Y , in addition to the complement C .

It can be inconvenient that the two branches must use the same complement. We can compromise and allow different complement types, sacrificing the ability to reuse the complement across branches (this only works because we allow **nothing** for missing complements):

```
_<+>_ : (p : Biparser Token U E C1 X Y)
  → (q : Biparser Token U E C2 X Y)
  →  $\llbracket$  ParsePrintConsistent (relaxCL p) (relaxCR q)  $\rrbracket$ 
  →  $\llbracket$  PrintParseConsistent (relaxCL p) (relaxCR q)  $\rrbracket$ 
  → Biparser Token U E (C1  $\uplus$  C2) X Y
p <+> q = relaxCL p <+> relaxCR q
```

The two constraints **PARSEPRINTCONSISTENT** and **PRINTPARSECONSISTENT** are necessary to ensure the round-trip properties for biased choices, and below we explain the reason by analysing the semantics for backtracking choices.

Always preferring the left branch is usually the desired behaviour, and this biased semantics is exploited to resolve ambiguity in the grammar. For example, a typical use of choice is to define the **optional_U** combinator:

```
optionalU : P Token U E X → P Token U E (Maybe X)
optionalU p = (just  $\ltimes$  p) <|> pure nothing
```

The parser **optional_U** p is greedy because the left branch is attempted first, and therefore **optional_U** p will not produce a **nothing** when parsing p succeeds on the input. This greediness makes sure that **twice** = **liftA2** **_,_** (**optional_U** p) (**optional_U** p) never produces (**nothing**, **just** x), while the following grammar is ambiguous and can generate the above result:

```
 $\langle$ Twice $\rangle ::= \langle$ Optional $P\rangle \langle$ Optional $P\rangle$ 
 $\langle$ Optional $P\rangle ::= \langle$ P $\rangle \mid \epsilon$ 
```


However, greediness only resolves the ambiguity in the parser, and without special treatment, the printer remains ambiguous. Specifically, there is a potential danger of branch mismatch in a round-trip, which may end up violating the round-trip properties.

For example, the following definition is a direct translation of the `optionalU` combinator:²

$\text{optional}^{\Delta} : \text{Biparser } \text{Token } U \text{ String } C \top X \rightarrow \text{Biparser } \text{Token } U \text{ String } (C \uplus \top) \top (\text{Maybe } X)$
 $\text{optional}^{\Delta} p = \text{mkJust } \# \bullet p <+> \text{mkNothing } \# \bullet \text{id}$

where `mkJust` and `mkNothing` are from Example 3.1. This naïve definition violates the `PRINTPARSE` property (because the two branches violate the consistency conditions). Consider the extreme case where $p = \text{id}$: printing `nothing` succeeds because the printer falls back on the right branch of the choice, but `parse (optional id)` never produces a `nothing` because `id` always succeeds, or in other words, for the parser, the right branch is never taken.

For a real-world example, the problem about unquoted YAML strings mentioned in Section 1 is also essentially an ambiguity in the grammar. The same string can be interpreted as either an object or an unquoted string. Eliminating ambiguities in the grammar is complicated, and it is also unnecessary in combinator-based unidirectional parsers, because we can rely on the left branch to be preferred. However, as we have already explained, for biparsers with exact-printers, we must take grammar ambiguities seriously.

To rule out such illegal usage and make sure the round-trip properties are always preserved, our biased choice combinator requires additional consistency conditions between the two branches f and g . Intuitively, these conditions specify that a branch mismatch in round-trips is only allowed when both branches agree on the result.

$$\frac{\begin{array}{l} \text{runP } (\text{print } p \text{ } (y, mc)) s \equiv \text{err } e \\ \text{runP } (\text{print } q \text{ } (y, mc)) s \equiv \text{ok } (x, s') \\ \text{runP } (\text{parse } p \text{ } x) s' \equiv \text{ok } ((y', c'), s'') \end{array}}{y \equiv y' \quad s \equiv s'} \quad (\text{PRINTPARSECONSISTENT})$$

$$\frac{\begin{array}{l} \text{runP } (\text{parse } p \text{ } x) s \equiv \text{err } e \\ \text{runP } (\text{parse } q \text{ } x) s \equiv \text{ok } ((y, c), s') \\ \text{runP } (\text{print } p \text{ } (y, \text{just } c)) s' \equiv \text{ok } (x', s'') \end{array}}{x \equiv x' \quad s \equiv s'} \quad (\text{PARSEPRINTCONSISTENT})$$

To see how these consistency conditions help to justify that `<+>` preserves the round-trip properties, one can expand the definitions and perform a case analysis. The interesting cases are when different branches are taken for parsing and printing, and when it happens, the consistency conditions make sure the result remains the same as if the matching branch were taken instead.

We can further prove that the above consistency conditions are in fact logically equivalent to the round-trip properties of biased choices. However, they might not be very convenient to use, because of their long premises. Therefore, we present the following conditions with simpler but weaker premises (which means the conditions are stronger, and using these conditions will rule out some lawful biparsers):

$$\frac{\begin{array}{l} \text{runP } (\text{parse } p \text{ } x) s \equiv \text{ok } ((y_1, c_1), s_1) \\ \text{runP } (\text{parse } q \text{ } x) s \equiv \text{ok } ((y_2, c_2), s_2) \end{array}}{y_1 \equiv y_2 \quad s_1 \equiv s_2} \quad (\text{PARSECONSISTENT})$$

$$\frac{\begin{array}{l} \text{runP } (\text{print } p \text{ } (y, mc)) s \equiv \text{ok } (x_1, s_1) \\ \text{runP } (\text{print } q \text{ } (y, mc)) s \equiv \text{ok } (x_2, s_2) \end{array}}{x_1 \equiv x_2 \quad s_1 \equiv s_2} \quad (\text{PRINTCONSISTENT})$$

These simpler versions also offer a simpler intuition: where both parsers (or printers) succeed, they must agree on their output, to the extent required by the round-trip properties, i.e., only the complements produced by `parse` can be different, and the others must be equal. These conditions are trivially satisfied if no common input is accepted by both parsers (or printers).

Example 3.6. Here are some examples where the consistency conditions are or are not satisfied.

²This definition is incorrect, so we mark it with a warning sign (Δ).

- (1) Consider two biparsers for JSON strings and JSON integers. They never both succeed on the same input and their printer always produce distinct output. Therefore, they trivially satisfy the consistency conditions.
- (2) Consider two biparsers for verbatim and escaped characters in JSON strings. The input accepted by their parsers is different due to the existence or absence of the backslash, so **PARSECONSISTENT** is trivially satisfied. However, their printers both accept the data, and the printed results are always different (e.g., "a" v.s. "\u0061"). Therefore, even **PARSEPRINTCONSISTENT**, the weaker condition, is not satisfied.

3.3.3 Remembered Choice. The remembered choice is introduced to provide a different trade-off. It also prefers the left branch in the parser, but it remembers which branch it took in the complement, and prefers the same branch in the printer.

```

_<>_: (p : Biparser Token U E C1 X Y)
      → (q : Biparser Token U E C2 X Y)
      → { ParseConsistent p q }
      → Biparser Token U E (C1 ⊔ C2) X Y

parse (p <> q) x = (λ (y, c1) → (y, inj1 c1)) <$> (parse p x)
                  <|> (λ (y, c2) → (y, inj2 c2)) <$> (parse q x)

print (p <> q) (y, just (inj1 c1)) = print p (y, just c1) <|> print q (y, nothing)
print (p <> q) (y, just (inj2 c2)) = print q (y, just c2) <|> print p (y, nothing)
print (p <> q) (y, nothing)         = print p (y, nothing) <|> print q (y, nothing)

```

Unlike biased choice, the remembered choice does not always prefer the left branch in the printer, but guarantees that the same branch is used for any parse-print round-trip. Therefore, the **PARSEPRINT** property is satisfied even without the consistency conditions, but the **PRINTPARSE** property requires the stronger **PARSECONSISTENT** condition.

Example 3.7 (YAML boolean values). Assume that we have three biparsers *pTrue*, *pOn*, and *pYes* for the three written forms for **true** (true, on, and yes) in YAML. We need to construct a biparser for any **true** boolean values.

```

pTrue pOn pYes : Biparser Char U E T T T
yamlTrue : Biparser Char U E (T ⊔ T ⊔ T) T T
yamlTrue = pTrue <> pOn <> pYes

```

The printer must try to use the same text as in the original source code.

Using different choice combinators results in different behaviours of the biparser. We determine which variant to use by considering the semantics and the consistency conditions between the two branches. Failing to satisfy the **PARSEPRINTCONSISTENT** (or **PRINTCONSISTENT**) property usually indicates that the remembered choice is necessary.

Establishing consistency conditions. The remembered choice only requires **PARSECONSISTENT**, but it is possible that the two branches do not even satisfy that. As a last resort, we can establish the consistency conditions by using the following **notFollowedBy** combinator.

First, we define the **lookahead** biparser which verifies that a given parser succeeds without actually consuming the input in the parser or producing any output in the printer.

```

lookahead : (X → P Token U String Y) → Biparser Token U String T X X
runP (parse (lookahead p) x) s = (λ (y, _) → ((x, tt), s)) <$> runP (p x) s
runP (print (lookahead p) (x, _)) s = (λ (y, _) → (x, s)) <$> runP (p x) s

```

Then, we can invert the success condition for the lookahead parser and get `notFollowedBy`:

```
notFollowedBy : (X → P Token U E Y) → Biparser Token U String T X X
notFollowedBy p = lookahead (expectErr ∘ p)

expectErr : P Token U E X → P Token U String T
runP (expectErr p) s = check (runP p s)
  where check : Result E X → Result String (T × List Token × U)
        check (err e) = ok (tt, s)
        check (ok x) = err "notFollowedBy: lookahead parser succeeds"
```

The biparser `notFollowedBy p` only succeeds when the remainder string is not accepted by the parser `p`. Similar to `validate` in Section 3.1, the checks in `print` are necessary for the `PRINTPARSE` property, and as mentioned in Section 2.1, it is in turn essential that in the printer, the text following the current fragment being printed is presented as the input string for checking.

For any two biparsers `p` and `q` of the same type, we can wrap `q` as follows:

```
q' = q •# notFollowedBy (parse p)
```

Everything accepted by `parse p` is now rejected by `parse q'`, so `PARSECONSISTENT` is trivially satisfied by `p` and `q'`. However, this comes at a price. Since `notFollowedBy` works by rejecting some printer results, it effectively makes some complement-data combinations invalid (in the sense that they cannot be successfully printed to source text). Therefore, `notFollowedBy` should be avoided or at least used with special care. See the appendix in the artifact for detailed analysis.

Example 3.8. Simple iterations can be derived as follows. The printer will run out of complements when the `List X` is longer than `List C`. We want the printer to try and print anyway, so we print without a complement when the list of complements runs short.

```
many some : Biparser Token U String C T X
           → Biparser Token U String (List (Maybe C)) T (List X)
some p = mapC (λ (c, cs) → c :: cs) uncons (mkCons #• (relaxC p <*> many p))
  where uncons : List (Maybe C) → Maybe C × List (Maybe C)
        uncons (c :: cs) = (c, cs)
        uncons []       = (nothing, [])
many p = some p <+> notFollowedBy (parse p) #*> mapC (λ _ → []) (λ _ → tt) mkNil
```

The printers of `mkNil` and `some` accept different ASTs (empty and non-empty list), so they satisfy `PRINTCONSISTENT`. However, the printer of `mkNil` always succeeds and never produces the same output as `some` (the left branch), so they do not satisfy `PARSECONSISTENT`. We use `notFollowedBy` here to force the consistency (`p` is a prefix of the left branch).

As a special case, if the biparser `p` has `T` complement, there is no need to collect them into `List (Maybe T)`, and we can define the following slightly-modified versions instead:

```
many' some' : Biparser Token U String T T X → Biparser Token U String T T (List X)
some' p = mkCons #• (p #*> many' p)
many' p = some' p <+> notFollowedBy (parse p) #*> mkNil
```

We will see a lot of use cases for these combinators in Section 4.

Example 3.9 (Sequence with separators). Similar to `many` and `some`, we can also define the following biparser for sequence of items with the given separators.

```

sepBy1 : Biparser Token U String C  $\top$  X
  → Biparser Token U String Csep  $\top$   $\top$ 
  → Biparser Token U String (List (Maybe C  $\times$  Maybe Csep))  $\top$  (List X)
sepBy1 p sep = mkCons #• mapC wrap unwrap (relaxC p <*> rest)
  where empty = mapC ( $\lambda$  _ → (nothing, [])) ( $\lambda$  _ → tt) (const [] null)
        rest = relaxC sep #> sepBy1 p sep <+> notFollowedBy (parse (sep <*> p)) #> empty
        wrap (c, csep, cs) = (c, csep) :: cs
        unwrap [] = (nothing, nothing, [])
        unwrap ((c, csep) :: cs) = (c, csep, cs)

```

This combinator requires at least one element of p , hence the 1 in the name.

3.4 View-Complement Alignment

The repetition combinators like `many` and `some` have a list of some X as the output and a list of some C as the complement. Now it is time to take a closer look at how their printers associate each individual X with a corresponding C (we call this process the view-complement alignment).

Since `many`, `some`, and `sepBy1` are defined recursively, in each step of recursion, the printer takes the head element of the output list (in the printer of `mkCons`), and also the head element of the complement list (in the printer of `mapC`). Therefore, the views and the complements are paired up according to their positions in the list. In other words, they use *positional alignment* by default.

For practical applications, we need more than the default positional alignment. Therefore, we provide the following `align` combinator, which takes an `AlignStrategy`.

```

AlignStrategy : (Y C : Set) → Set
AlignStrategy Y C = Y → C → C

```

An `AlignStrategy` takes the modified view Y and the original complement C and tries to produce an *aligned* complement. For lists, being aligned means that the complements are at the same positions as their corresponding views, so that the default positional alignment would result in the expected view-complement matching.

```

align : (f : AlignStrategy Y C) → (p : Biparser Token U String C X Y)
  →  $\llcorner$  AlignConsistent f p  $\lrcorner$  → Biparser Token U String C X Y
parse (align f p) x      = parse p x
print (align f p) (y, c) = print p (y, f y  $\diamond$  c)

```

This `align` combinator applies an alignment strategy f before printing with p , in the hope that the new complement matches the internal structure of Y better. For the round-trip properties of `align`, we require that if the view is unchanged, the align strategy f should also leave the complement unchanged, i.e., return the same original complement (`AlignConsistent`).

We intend to use the `align` combinator together with the repetition combinators, whose views and complements are both lists. In such cases, just like *Matching Lenses* [Barbosa et al. 2010], we have the following possible alignment strategies specific to lists:

- Positional: this strategy coincides with the default behaviour without explicit `align`, which leaves the complement unchanged.

```

positional : AlignStrategy (List Y) (List (Maybe C))
positional _ cs = cs

```

- By key: this strategy tries to extract a key K from both view entries Y and complement entries C , and use equality of K to match C against Y .

```
byKey : (Y → K) → (C → K) → AlignStrategy (List Y) (List (Maybe C))
byKey yKey cKey ys cs = map (λ y → Map.lookup (yKey y) kcs) ys
  where kcs = Map.fromList (map (λ c → (cKey c, c)) (catMaybes cs))
```

Depending on the properties of the key type K , the `Map` can be an ordered map or a hashmap. This strategy is suitable for JSON and YAML objects: we can use the `store` combinator to remember the string key as part of the complement for each entry, and use that string key to perform fast perfect matching.

- Best match: if we have a good distance function $d : Y \times C \rightarrow \mathbb{R}$, we can build a bipartite graph between `List Y` and `List C`, where the edge between $y : Y$ and $c : C$ has weight $d(y, c)$. In this way, the best alignment of views and complements can be reduced to the least-weighted match in said bipartite graph, which is a solved problem. This strategy is suitable for small JSON and YAML arrays, because it tends to give high-quality results, but may also take a long time.
- Best non-crossing match: similar to best match, but with order of elements preserved, used by the `diff` program for text documents. This type of strategy is also suitable for JSON and YAML arrays; it may perform better than plain best match, but meanwhile it may fail to reuse some complements, leading to the loss of formatting.
- Actual edit: if the edit is done in a text editor or generated by a program, then it is possible to use the actual edit operations to guide the alignment. This strategy is not generally applicable, but when it can be applied, it gives the most accurate result with little performance penalty.

The alignment strategy is worth its own research, and the use of alignment strategies is orthogonal to other parts of biparsers. Besides, the choice of a specific alignment strategy should always be based on the characteristics of the data and the complement. Therefore, we do not dive into the details in this paper.

4 Examples: JSON and YAML

In this section, we develop a biparser for subsets of JSON and YAML to show the usefulness of our combinators, and to demonstrate the development methodology of biparsers. We start with the building blocks and assemble them into the final biparser in the end. The implementation in Haskell can be found in the artifact (around 200 lines of code for each format).

4.1 Biparser for JSON

Fig. 2a shows the grammar subset we consider for JSON, written in EBNF, where $\langle X \rangle$ is a non-terminal symbol X , `x` is a literal string, and `x` is a terminal symbol; $x?$ means an optional x , x^* means zero or more of x , x^+ means one or more of x , and $x\{n\}$ means x repeated n times. The structure of the biparsers follow the grammar rules closely.

Whitespace. Whitespace can appear anywhere in a JSON document, so we start with the biparser for whitespace, which we represent as `Whitespace = List Char`.

```
spaces : Biparser Char U String T T Whitespace
spaces = many' (satisfy (λ c → elem c " \n\r\t"))
```

The `spaces` combinator consumes zero or more whitespace characters, because whitespace is completely optional in JSON documents. We also define `white = drop [] •# spaces`.

$\langle \text{Value} \rangle ::= \langle \text{String} \rangle \mid \langle \text{Number} \rangle \mid \langle \text{Object} \rangle$ $\mid \langle \text{Array} \rangle \mid \text{true} \mid \text{false} \mid \text{null}$ $\langle \text{Number} \rangle ::= 0 \mid - ? \text{NONZERO} \text{DIGIT}^*$ $\langle \text{String} \rangle ::= " \langle \text{Char} \rangle^* "$ $\langle \text{Char} \rangle ::= \text{NONCONTROL} \mid \backslash \langle \text{CEsc} \rangle \mid \backslash \langle \text{HexEsc} \rangle$ $\langle \text{CEsc} \rangle ::= " \mid \backslash \mid / \mid b \mid f \mid n \mid r \mid t$ $\langle \text{HexEsc} \rangle ::= u \text{HEX} \{4\}$ $\langle \text{Object} \rangle ::= \{ \langle \text{Members} \rangle \}$ $\langle \text{Members} \rangle ::= \langle \text{Member} \rangle \mid \langle \text{Members} \rangle , \langle \text{Member} \rangle$ $\langle \text{Member} \rangle ::= \langle \text{String} \rangle : \langle \text{Value} \rangle$ $\langle \text{Array} \rangle ::= [\langle \text{Elements} \rangle]$ $\langle \text{Elements} \rangle ::= \langle \text{Value} \rangle \mid \langle \text{Elements} \rangle , \langle \text{Value} \rangle$	$\langle \text{Value}_n \rangle ::= \langle \text{String} \rangle \mid \langle \text{Number} \rangle \mid \langle \text{Object}_n \rangle$ $\mid \langle \text{Array}_n \rangle \mid \langle \text{True} \rangle \mid \langle \text{False} \rangle \mid \text{null}$ $\langle \text{Number} \rangle ::= 0 \mid - ? \text{NONZERO} \text{DIGIT}^*$ $\mid 0 \text{o} \text{OCT}^+ \mid 0 \text{x} \text{HEX}^+$ $\langle \text{String} \rangle ::= " \langle \text{Char} \rangle^* " \mid \text{PLAINCHAR}^+$ $\langle \text{Char} \rangle ::= \text{NONCONTROL} \mid \backslash \langle \text{CEsc} \rangle \mid \backslash \langle \text{HexEsc} \rangle$ $\langle \text{CEsc} \rangle ::= " \mid \backslash \mid / \mid b \mid f \mid n \mid r \mid t$ $\mid 0 \mid a \mid v \mid e \mid _$ $\langle \text{HexEsc} \rangle ::= x \text{HEX} \{2\} \mid u \text{HEX} \{4\} \mid U \text{HEX} \{8\}$ $\langle \text{Object}_n \rangle ::= (\text{INDENT}_n \langle \text{Member}_n \rangle)^*$ $\langle \text{Member}_n \rangle ::= \langle \text{String} \rangle : _ \langle \text{Value}_m \rangle \quad (m > n)$ $\langle \text{Array}_n \rangle ::= (\text{INDENT}_n \langle \text{Element}_n \rangle)^*$ $\langle \text{Element}_n \rangle ::= - _ \langle \text{Value}_m \rangle \quad (m > n)$ $\langle \text{True} \rangle ::= \text{true} \mid \text{on} \mid \text{yes}$ $\langle \text{False} \rangle ::= \text{false} \mid \text{off} \mid \text{no}$
---	---

(a) JSON Grammar

(b) YAML Grammar

Fig. 2. Grammars for the JSON and YAML Subsets

In their overview of design patterns for parser combinators, Willis and Wu [2021] suggest that each parser is responsible for consuming the trailing whitespace. In addition, they provide a dedicated combinator **fully** to consume the initial whitespace and the end of input. Similarly, we define a **fully** combinator as well as the **delim** biparser for delimiters with trailing whitespace, which is a common situation for JSON (curly brackets, square brackets, colons, commas, etc.):

fully : Biparser Char U String $C \top X \rightarrow$ Biparser Char U String (Whitespace $\times C$) $\top X$

fully $p = \text{white} \gg p \ll \text{eof}$

delim : Char $\rightarrow$ Biparser Char U String Whitespace $\top \top$

delim $c = \text{char } c \# \ll \text{white}$

Boolean and Null Values. These values are written literally as true, false, and null.

jsonBool : Biparser Char U String Whitespace $\top$ Bool

jsonBool = (const' true $\bullet \#$ string "true" $<+>$ const' false $\bullet \#$ string "false") $\# \ll \text{white}$

jsonNull : Biparser Char U String Whitespace $\top \top$

jsonNull = string "null" $\# \ll \text{white}$

Numbers. In our subset of JSON, we only consider integers. According to the JSON specification, non-zero numbers cannot have leading zeros and zero cannot have a minus sign. In other words, an integer is either a single zero digit or a non-zero digit followed by zero or more digits. We start with biparsers for digits and non-zero digits.

digit nonZero : Biparser Char U String $\top \top \mathbb{N}$

digit = partialIso digitToInt intToDigit $\# \bullet$ item

nonZero = validate $(\lambda n \rightarrow n \neq 0) \# \bullet$ digit

We map the optional minus sign to a boolean flag for negativity, where the two branches satisfy the consistency conditions for choices thanks to the use of the **notFollowedBy** combinator.


```

minusSign : Biparser Char U String T T Bool
minusSign = const' true #• char '-' <+> const' false #• notFollowedBy (parse (char '-'))

```

Finally, we combine these biparsers and get the biparser for integers. The structure of this biparser follows the grammar closely, with one branch for zero and another branch for non-zero (positive or negative) numbers.

```

jsonNumber : Biparser Char U String Whitespace T T ℤ
jsonNumber = (zero <+> other) #• white
  where zero = const' 0ℤ #• char '0'
        other = makeNumber #• (minusSign #<*> nonZero #<*> many' digit)

```

Here `mkNumber` is a partial isomorphism for non-zero numbers:

```

makeNumber : Biparser Char U String T (Bool × ℕ × List ℕ) ℤ

```

The two branches satisfy the consistency conditions for choices trivially, because their inputs and outputs are both disjoint.

Strings. A JSON string is zero or more escaped or non-escaped characters surrounded by a pair of double quotes (the complement is determined by the complement for the `stringChar` biparser, which we will explain in the end):

```

jsonString : Biparser Char U String (List (Maybe EscapeFormat) × Whitespace) T (List Char)
jsonString = (char '"' #*> many stringChar <*># char '"') <*> white

```

Now we only need to consider the string contents.

According to the grammar, there are the normal non-control characters, which can be included in strings verbatim. For simplicity, we allow everything other than double quote (for string boundaries) and backslash (for escape sequences). We define the biparser using a predicate:

```

nonControl : Biparser Char U String T T Char
nonControl = satisfy (λ c → c /= '\\' ∧ c /= '"')

```

Besides these, there are the C-style escape sequences for a selected collection of control characters. We define the biparser by enumerating all the possibilities. The consistency conditions for choices are trivially satisfied because their inputs and outputs are disjoint.

```

cEscape : Biparser Char U String T T Char
cEscape =  const' '"' #• char '"'
          <+> const' '\\' #• char '\\'
          <+> const' '/' #• char '/'
          <+> const' '\b' #• char 'b'
          <+> const' '\f' #• char 'f'
          <+> const' '\n' #• char 'n'
          <+> const' '\r' #• char 'r'
          <+> const' '\t' #• char 't'

```

Unlike C-style escape sequences, the Unicode escape sequences are a general mechanism for representing any Unicode character between U+0000 and U+FFFF, so it is impractical to enumerate every possibility. Instead, we sequentially parse four hex-digits and transform the quadruple into the resulting character.

```

unicodeEscape : Biparser Char U String (Bool × Bool × Bool × Bool) T Char
unicodeEscape = char 'u' #*> (makeHexChar #• quadruple)
  where quadruple = hexDigit <*> hexDigit <*> hexDigit <*> hexDigit

```

The `hexDigit` biparser accepts both uppercase and lowercase hex-digits. This information is not reflected in its output, so we use a boolean complement to remember it (in case the hex-digit is not a letter, we may default to either).

```
hexDigit : Biparser Char U String Bool T N
```

The `makeHexChar` biparser is responsible for transforming between the quadruple of hex-digits and the Unicode character. For simplicity, we reject Unicode code points greater than 0xFFFF in the backward transformation.

```
makeHexChar : Biparser Char U String T (N × N × N × N) Char
```

Now we combine the three possible formats and get the biparser for characters in JSON strings. The accepted inputs are disjoint, but their valid outputs are not. For example, a (accepted by `nonControl`) and `\u0061` (accepted by `unicodeEscape`) both parse into the lowercase letter “a”; `\n` (accepted by `cEscape`) and `\u000A` (accepted by `unicodeEscape`) both parse into the line feed character. Therefore, we must use the remembered choice.

```

stringChar : Biparser Char U String EscapeFormat T Char
stringChar = mapC wrapEscapeFormat unwrapEscapeFormat
  (nonControl </> (char '\\ ' #*> (cEscape </> unicodeEscape)))

```

For readability, we wrap the complement into the `EscapeFormat` data type:

```

data EscapeFormat where
  NonControl : EscapeFormat
  CEscape : EscapeFormat
  UnicodeEscape : (a b c d : Bool) → EscapeFormat

wrapEscapeFormat : T ⊔ T ⊔ Bool × Bool × Bool × Bool → EscapeFormat
unwrapEscapeFormat : EscapeFormat → T ⊔ T ⊔ Bool × Bool × Bool × Bool

```

Notice that the original complement is a sum of products, which maps cleanly to algebraic data types like `EscapeFormat`. In general, we do not design the complement type. Instead, the shape of the complement follows from the biparser structure.

Arrays. A JSON array is a comma-separated list of JSON values surrounded by a pair of square brackets. We define the biparser and its complement type as follows:

```

CArray = Whitespace × List (Maybe CValue × Maybe Whitespace) × Whitespace

jsonArray : Biparser Char U String CArray T (List Value)
jsonArray = delim '[' *> sepBy1 jsonValue (delim ',' ) <*> delim ']'

```

Objects. A JSON object is a comma-separated list of key-value pairs surrounded by a pair of curly brackets. Similarly, we define the biparser and its complement type as follows:

```

CEntry = ((List (Maybe EscapeFormat) × Whitespace) × Whitespace × CValue) × List Char
CObject = Whitespace × List (Maybe CEntry × Maybe Whitespace) × Whitespace

jsonObject : Biparser Char U String CObject T (List (List Char × Value))

```

```

jsonObject = delim '{' *> sepBy1 entry (delim ',') <*> delim '}'
  where entry = store proj1 • (jsonString <*> delim ':' *> jsonValue)

```

Values. Finally, a JSON value is any one of the above possibilities. The inputs and outputs are disjoint, so the consistency conditions of the choices are again trivially satisfied.

```

jsonValue : Biparser Char U String CValue T Value
jsonValue = mapC wrapCValue unwrapCValue
  $ mkNumber #• jsonNumber
  <+> mkString #• jsonString
  <+> mkObject #• jsonObject
  <+> mkArray #• jsonArray
  <+> mkBool #• jsonBool
  <+> mkNull #• jsonNull

```

The complement type for JSON values is the following data type, which maps to and from the original sum-of-products complement type with `wrapCValue` and `unwrapCValue`.

```

data CValue where
  cString : List (Maybe EscapeFormat) × Whitespace → CValue
  cObject : CObject → CValue
  cArray : CArray → CValue
  cNumber cBool cNull : Whitespace → CValue

```

The biparser `fully value` is our final biparser for a full JSON document.

4.2 Biparser for YAML

The YAML grammar is quite similar to the JSON grammar. In case of numbers and strings, YAML merely provides several formatting options, so we do not repeat ourselves for these biparsers. However, there is a unique characteristic of YAML, which is indentation sensitivity. In this section, we focus on how we implement the biparsers for YAML indentations.

Before we start, we would like to point out that it is likely more flexible and more efficient to preprocess leading whitespace into indent tokens with a separate tokenisation pass. However, the purpose of this example is to illustrate how the user state U can be used to handle context sensitivity, which is orthogonal to the tokenisation pass. Therefore, we stick with this straightforward approach.

We define a biparser for parsing leading whitespace into an indentation level.

```

level : Biparser Char (List ℤ) String (List Char) T ℤ

```

In an object or an array, the indentation level increases (relative to the parent) before the first entry, and every following entry keeps the same increased indentation. Therefore, we can use the state $U = \text{List } \mathbb{Z}$ to maintain an indentation stack, define operations in the $\mathbf{P}$ monad to query the current indentation, push onto and pop from the stack:

```

getIndentLevel : P Token (List ℤ) E ℤ
pushIndent : ℤ → P Token (List ℤ) E T
popIndent : P Token (List ℤ) String T

```

These operations can in turn be used to define the following two biparsers for checking an indentation level k against the current level n on the stack: `inc` expects $k > n$ and pushes k onto the stack, while `eq` expects $k = n$ and leaves the stack untouched.

`inc eq : Biparser Char (List  $\mathbb{Z}$ ) String  $\top \mathbb{Z} \top$`

Besides, we have a biparser for unconditionally popping one level of indentation off the stack:

`dedent : Biparser Char (List  $\mathbb{Z}$ ) String  $\mathbb{Z} \top \top$`

Finally, using `indent = inc #• level`, `keep = eq #• level`, and the `dedent` above, we can correctly handle block constructs with any entry p :

```
block : Biparser Char (List  $\mathbb{Z}$ ) String  $C \top X$ 
      → Biparser Char (List  $\mathbb{Z}$ ) String (List (Maybe (List Char  $\times$  C))  $\times \mathbb{Z}$ )  $\top$  (List X)
block p = mkCons #• mapC to from (relaxC (indent  $\gg$  p)  $\ll$  many (keep  $\gg$  p))  $\ll$  dedent
  where to =  $\lambda (c, cs) \rightarrow c :: cs$ 
        from : List (Maybe C)  $\rightarrow$  Maybe C  $\times$  List (Maybe C)
        from [] = (nothing, [])
        from (c :: cs) = (c, cs)
```

This `block` combinator can be used to define biparsers for objects and arrays.

5 Related Work

Besides the related work discussed in the introduction, our work is very much related to the work on bidirectional parsers and bidirectional programming.

5.1 Bidirectional Parsers

Bidirectional parsers derive a parser and a printer simultaneously from a single definition. The parser and the printer are either directly constructed from combinators, or generated from grammar descriptions.

Arrow-Based Combinators. Bi-arrows [Alimarine et al. 2005] are a direct generalisation of the (unidirectional) arrow interface [Hughes 2000], with parsers and pretty-printers as one of its use cases. Invertible syntax descriptions [Rendel and Ostermann 2010] are designed to resemble parser combinators based on applicative functors, but the similarity is actually only in appearance: as we have illustrated in Section 3.2, the applicative-style combinators can be defined using the arrow-style combinators. We believe that the syntactic difference is inevitable in a bidirectional context, yet is not a critical issue for the user.

Monadic Combinators. The monadic interface is still the most familiar interface for parser combinators, and monadic profunctors have been proposed [Xia et al. 2019] so that biparsers can be written in a monadic structure, but with additional *comap* annotations. However, their round-trip properties are weaker due to a property which is not compositional (the purification property); it must be verified separately for every biparser, and therefore their biparsers are not *correct by construction*. Besides, their biparser example focuses on pretty-printing, and it remains unclear how their biparsers and their lenses can be combined to achieve exact-printing.

DSLs and Generators. It is possible to generate both a parser and a pretty-printer from a single annotated grammar [Boulton 1996]. FliPpr [Matsuda and Wang 2018] allows more fine-grained control over pretty-printing: it starts with pretty-printer written in a DSL, extracts a grammar description, and then delegates to a parser generator for the parser. BiYacc [Zhu et al. 2020] uses the putback-based bidirectional programming style [Ko et al. 2016], and the user program is limited to structurally recursive updates. These generator-based approaches require more effort to migrate an existing parser written using parser combinator libraries like Parsec. In comparison, our approach is based on combinators, and we support an internal user state U , which enables

context sensitivity in the grammar (e.g., indentations) and stateful transformations of the parsed data structures. Boomerang [Bohannon et al. 2008] is a DSL for lenses between strings. It does not support representing the data as an algebraic data type (ADT), and its expressive power is limited because it prohibits ambiguity in concatenations and unions. In comparison, our backtracking choice combinators resolve ambiguity in a more permissive way.

5.2 Bidirectional Programming

Bidirectional programming develops programs that transform between two different data types. Existing approaches define various correctness properties and provide tools to enforce these properties.

Invertible Programming. Bi-arrows [Alimarine et al. 2005] and partial isomorphisms [Rendel and Ostermann 2010] are bidirectional programming models based on invertibility. These approaches assume that transforming between different data representations is lossless in information, and therefore the backward computation of a bidirectional program can be considered as the inverse of the forward computation. However, due to the information loss in most parsers, this model can only cover the print-parse round-trip. In addition, the left-invertibility for bi-arrows is also overly restrictive, such that their example parser [Alimarine et al. 2005, Section 7] is not invertible.

Lenses. Lenses [Foster et al. 2007] are a more powerful model for bidirectional programming that takes the information loss into account. The idea of view complements comes from relational updates in the database literature [Bancilhon and Spyrtos 1981] and is used as a basis for symmetric lenses [Hofmann et al. 2011], which is a generalisation of (asymmetric) lenses. It should be possible to base our biparsers on symmetric lenses, but due to the structure of parsers and printers, information loss rarely happens in the printer, so we believe asymmetric lenses are a better fit for the biparser semantics.

Monadic Lenses. Monadic lenses aim to combine lens frameworks with monadic effects. However, previous works on monadic lenses [Abou-Saleh et al. 2015; Pacheco et al. 2014] have serious limitations. As summarised by Abou-Saleh et al. [2016], in order for lens composition to preserve the round-trip properties, asymmetric monadic lenses only allow effects in *put*, and symmetric monadic lenses must be represented as spans of asymmetric monadic lenses. For our purpose, both *get* and *put* have effects, so we could not reuse the existing asymmetric monadic lenses. For symmetric monadic lenses, their round-trip properties are too permissive, i.e., they pose no requirements on the relationship between the effects happening in *get* and *put*, so we also could not reuse them. Besides, none of the existing approaches provide a backtracking choice operator, which is crucial to parser combinator frameworks.

6 Conclusion

In this paper, we proposed a bidirectional parser framework for exact-printing of abstract syntax trees. We defined biparsers and their round-trip properties. These round-trip properties ensure the consistency between the parser and the printer: a piece of data printed as text can be parsed back to the same data (its meaning is not changed), and source text parsed into data can be recovered exactly. We also defined various biparser combinators for common patterns in biparser development: token consumption, result transformation, sequencing, choices, chains, etc. Finally, we used our combinators to construct a biparser for subsets of JSON and YAML. These biparsers correctly handle whitespace, comments, multi-format data, and indentations, which illustrates the expressivity of our framework.

Data-Availability Statement

The artifact for this paper is available online [Xie et al. 2024]. It contains the complete Agda proof, the Haskell library with the examples for subsets of JSON and YAML, and the appendix of this paper. In the appendix, we provide a summary of Agda notations used in this paper, definitions omitted for brevity, and a short discussion on why the `notFollowedBy` combinator should be used with special care.

Acknowledgments

We thank all the anonymous reviewers of this paper for their invaluable feedback. This work was partly supported by the National Natural Science Foundation of China (NSFC) under Grant W2411051, the Flemish Fund for Scientific Research (FWO) under project number 3E231241, and the Chinese Scholarship Council (CSC).

References

- Faris Abou-Saleh, James Cheney, Jeremy Gibbons, James McKinna, and Perdita Stevens. 2015. *Notions of Bidirectional Computation and Entangled State Monads*. Springer International Publishing, 187–214. https://doi.org/10.1007/978-3-319-19797-5_9
- Faris Abou-Saleh, James Cheney, Jeremy Gibbons, James McKinna, and Perdita Stevens. 2016. Reflections on Monadic Lenses. Springer International Publishing, Cham, 1–31. https://doi.org/10.1007/978-3-319-30936-1_1
- Artem Alimarine, Sjaak Smetsers, Arjen van Weelden, Marko van Eekelen, and Rinus Plasmeijer. 2005. There and back again: arrows for invertible programming. In *Proceedings of the 2005 ACM SIGPLAN workshop on Haskell (Haskell '05)*. Association for Computing Machinery, New York, NY, USA, 86–97. <https://doi.org/10.1145/1088348.1088357>
- F. Bancilhon and N. Spyrtos. 1981. Update semantics of relational views. *ACM Transactions on Database Systems* 6, 4 (1981), 557–575. <https://doi.org/10.1145/319628.319634>
- Davi M.J. Barbosa, Julien Cretin, Nate Foster, Michael Greenberg, and Benjamin C. Pierce. 2010. Matching lenses: alignment and view update. In *Proceedings of the 15th ACM SIGPLAN international conference on Functional programming (ICFP '10)*. Association for Computing Machinery, New York, NY, USA, 193–204. <https://doi.org/10.1145/1863543.1863572>
- Aaron Bohannon, J. Nathan Foster, Benjamin C. Pierce, Alexandre Pilkiewicz, and Alan Schmitt. 2008. Boomerang: resourceful lenses for string data. *ACM SIGPLAN Notices* 43, 1 (Jan. 2008), 407–419. <https://doi.org/10.1145/1328897.1328487>
- Richard J. Boulton. 1996. *Syn: a single language for specifying abstract syntax trees, lexical analysis, parsing and pretty-printing*. Technical Report UCAM-CL-TR-390. University of Cambridge, Computer Laboratory. <https://doi.org/10.48456/tr-390>
- Bryce Clarke, Derek Elkins, Jeremy Gibbons, Fosco Loregian, Bartosz Milewski, Emily Pillmore, and Mario Román. 2022. *Profunctor Optics, a Categorical Update*. Technical Report. <https://doi.org/10.48550/arXiv.2001.07488> arXiv:2001.07488 [cs, math] version: 2 type: article.
- J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. 2007. Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem. *ACM Transactions on Programming Languages and Systems* 29, 3 (2007), 17–es. <https://doi.org/10.1145/1232420.1232424>
- Martin Hofmann, Benjamin Pierce, and Daniel Wagner. 2011. Symmetric lenses. In *Proceedings of the 38th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages (POPL '11)*. Association for Computing Machinery, New York, NY, USA, 371–384. <https://doi.org/10.1145/1926385.1926428>
- John Hughes. 2000. Generalising monads to arrows. *Science of Computer Programming* 37, 1-3 (2000), 67–111. [https://doi.org/10.1016/S0167-6423\(99\)00023-4](https://doi.org/10.1016/S0167-6423(99)00023-4)
- Edward A. Kmett. 2024. *lens: Lenses, Folds and Traversals*. <https://hackage.haskell.org/package/lens>
- Hsiang-Shang Ko, Tao Zan, and Zhenjiang Hu. 2016. BiGUL: a formally verified core language for putback-based bidirectional programming. In *Proceedings of the 2016 ACM SIGPLAN Workshop on Partial Evaluation and Program Manipulation (PEPM '16)*. Association for Computing Machinery, New York, NY, USA, 61–72. <https://doi.org/10.1145/2847538.2847544>
- Daan Leijen and Erik Meijer. 2001. Parsec: Direct Style Monadic Parser Combinators for the Real World. (July 2001). <https://www.microsoft.com/en-us/research/publication/parsec-direct-style-monadic-parser-combinators-for-the-real-world/>
- John MacFarlane. 2024. *pandoc: Conversion between markup formats*. <https://hackage.haskell.org/package/pandoc>
- Kazutaka Matsuda and Meng Wang. 2018. FliPpr: A System for Deriving Parsers from Pretty-Printers. *New Generation Computing* 36, 3 (2018), 173–202. <https://doi.org/10.1007/s00354-018-0033-7>
- Conor McBride and Ross Paterson. 2008. Applicative programming with effects. *Journal of Functional Programming* 18, 1 (Jan. 2008), 1–13. <https://doi.org/10.1017/S0956796807006326>

- Ulf Norell. 2007. Towards a practical programming language based on dependent type theory. <https://www.cse.chalmers.se/~ulfn/papers/thesis.html>
- Ulf Norell. 2009. Dependently Typed Programming in Agda. Springer, Berlin, Heidelberg, 230–266. https://doi.org/10.1007/978-3-642-04652-0_5
- Hugo Pacheco, Zhenjiang Hu, and Sebastian Fischer. 2014. Monadic combinators for “Putback” style bidirectional programming. In *Proceedings of the ACM SIGPLAN 2014 Workshop on Partial Evaluation and Program Manipulation (PEPM ’14)*. Association for Computing Machinery, New York, NY, USA, 39–50. <https://doi.org/10.1145/2543728.2543737>
- Tillmann Rendel and Klaus Ostermann. 2010. Invertible syntax descriptions: unifying parsing and pretty printing. *ACM SIGPLAN Notices* 45, 11 (2010), 1–12. <https://doi.org/10.1145/2088456.1863525>
- Philip Wadler. 1995. Monads for functional programming. In *Advanced Functional Programming (Lecture Notes in Computer Science)*, Johan Jeuring and Erik Meijer (Eds.). Springer, Berlin, Heidelberg, 24–52. https://doi.org/10.1007/3-540-59451-5_2
- Jamie Willis and Nicolas Wu. 2021. Design patterns for parser combinators (functional pearl). In *Proceedings of the 14th ACM SIGPLAN International Symposium on Haskell (Haskell 2021)*. Association for Computing Machinery, New York, NY, USA, 71–84. <https://doi.org/10.1145/3471874.3472984>
- Li-yao Xia, Dominic Orchard, and Meng Wang. 2019. Composing Bidirectional Programs Monadically. In *Programming Languages and Systems (Lecture Notes in Computer Science)*, Luís Caires (Ed.). Springer International Publishing, Cham, 147–175. https://doi.org/10.1007/978-3-030-17184-1_6
- Ruifeng Xie, Tom Schrijvers, and Zhenjiang Hu. 2024. *Artifact for POPL ’25: Biparsers: Exact Printing for Data Synchronisation*. <https://doi.org/10.5281/zenodo.13939727>
- Zirun Zhu, Hsiang-Shang Ko, Yongzhe Zhang, Pedro Martins, João Saraiva, and Zhenjiang Hu. 2020. Unifying Parsing and Reflective Printing for Fully Disambiguated Grammars. *New Generation Computing* 38, 3 (April 2020), 423–476. <https://doi.org/10.1007/s00354-019-00082-y>

Received 2024-07-11; accepted 2024-11-07